

Full Stack Web Application (Node.js + Express + MongoDB)

This project is a full-stack web application built with **Node.js**, **Express**, **MongoDB (Mongoose)**, and **EJS**. It follows a clean **MVC (Model-View-Controller)** architecture and includes authentication, session management, product catalog display, and cart functionality.

Features

- User authentication (register, login, logout)
 - Secure password hashing with bcrypt
 - Session-based authentication
 - MongoDB database with Mongoose models
 - Product catalog rendering
 - Shopping cart persistence per user
 - Clean MVC folder structure
 - Environment variable configuration with dotenv
 - EJS templating with layouts
-

Project Structure

```
Full Stack Web Application
|
├─ app.js
├─ package.json
├─ .env
|
├─ config
|   └─ db.js
|   └─ session.js
|
├─ controllers
|   └─ authController.js
|   └─ catalogController.js
|   └─ cartController.js
|   └─ pageController.js
|
├─ middlewares
|   └─ authMiddleware.js
|   └─ globalVars.js
```

```
|
|
|--- models
|   |--- User.js
|   |--- Catalog.js
|   |--- CartItem.js
|
|--- routes
|   |--- authRoutes.js
|   |--- catalogRoutes.js
|   |--- cartRoutes.js
|   |--- pageRoutes.js
|
|--- public
|   |--- (static assets: CSS, JS, images)
|
|--- views
|   |--- (EJS templates)
```

Technologies Used

- Node.js
- Express.js
- MongoDB Atlas
- Mongoose
- EJS & express-ejs-layouts
- bcryptjs
- express-session
- dotenv

Environment Variables

Create a `.env` file in the project root:

```
MONGO_URI=mongodb+srv://<username>:<password>@cluster.mongodb.net/databaseName
SESSION_SECRET=yourSecretKey
NODE_ENV=development
```

 **Never commit `.env` to version control**

Installation & Setup

Clone the repository

```
git clone <repository-url>  
cd Full\ Stack\ Web\ Application
```

Install dependencies

```
npm install
```

Run the application

```
node app.js
```

Server will start on:

```
http://localhost:3000
```

MVC Architecture Explained

Models (/models)

- Define database schemas
- Handle MongoDB data structure

Controllers (/controllers)

- Contain business logic
- Handle requests and responses

Routes (/routes)

- Map URLs to controllers

Views (/views)

- EJS templates rendered to the client

Middlewares (/middlewares)

- Authentication checks

- Global variables for views
-

Authentication Flow

1. User registers with a hashed password
 2. User logs in
 3. Session is created and stored
 4. Protected routes use `authMiddleware`
 5. User logs out and session is destroyed
-

Cart System

- Cart items are stored per user
 - Cart data is sent via API
 - Items persist across sessions
-

Common Scripts

```
node app.js      # Run server
npm install      # Install dependencies
```

Future Improvements

- JWT-based authentication
 - Role-based access (admin/user)
 - MongoDB session store (connect-mongo)
 - Input validation (Joi / Zod)
 - REST API separation
 - Deployment (Render / Railway / Fly.io)
-

Learning Goals Achieved

- Understanding MVC architecture
 - Environment-based configuration
 - Secure authentication practices
 - Modular backend design
 - Debugging real-world Node.js issues
-

Author

Uthman Okunola

Software Developer | Information Systems

License

This project is for educational purposes.